UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 11 — Circuit breakers, backpressure, bulkheads, retries y graceful degradation

Departamento de Computación

Ingeniería Informática – Comisión A – Pilar

Equipo docente – 2026

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Apertura: de la observación a la contención

Sistemas Distribuidos

Dónde nos quedamos

La clase pasada terminó con la cascada de cuatro pasos:

1. Una base de datos aguas abajo se degrada.
2. El servicio intermedio agota su pool de hilos.
3. Los clientes aguas arriba **reintentan** y amplifican el tráfico.
4. El borde empieza a devolver HTTP 504.

Ya sabemos *ver* esa cascada. Esta clase: qué hace el sistema **mientras** ocurre.

El retry

En clases anteriores era la solución a un mensaje perdido. Frente a una dependencia saturada, es el combustible.

Falta, error y falla



- La *grieta* avanza de un eslabón al siguiente; el acoplamiento la acelera.
- Las dependencias **van** a fallar. Lo que se diseña es **dónde se corta**, como las zonas de deformación de un auto.
- "Failure is in the eye of the beholder": para un monitoreo que solo mira una página de estado, un servicio con todos los hilos bloqueados está sano.

Quioscos de *check-in*: terminales de autoservicio de los aeropuertos. Nygard, *Release It!* (2018), cap. 3; ejemplos del caso de la aerolínea (cap. 2). Apunte: "Introducción".

UNIVERSIDAD AUSTRAL

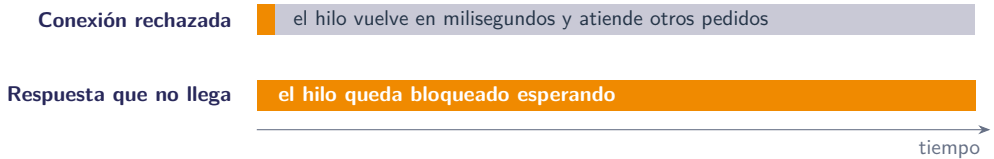
INGENIERÍA

Departamento de Computación

Anatomía de una falla en cascada

Sistemas Distribuidos

Falla rápida y falla lenta



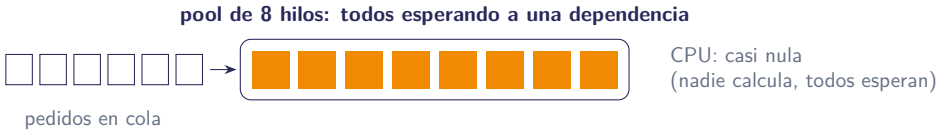
Nygard, cap. 4

"Clearly, a slow response is a lot worse than no response."

Una dependencia **lenta** es más peligrosa que una **caída**: la caída libera los recursos del llamante; la lentitud los retiene.

Nygard, *Release It!* (2018), cap. 4. Apunte: "Falla rápida y falla lenta".

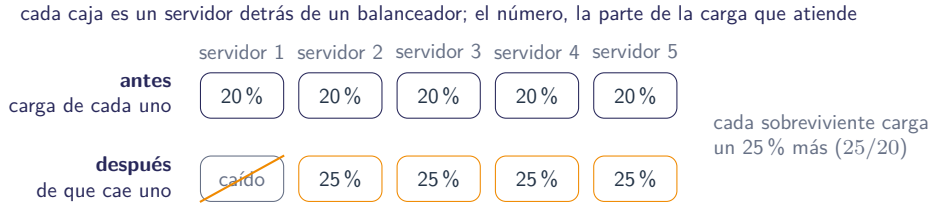
Hilos bloqueados: el pool se agota



- Casi ninguna caída real es un proceso que termina: es un proceso vivo con todos sus hilos esperando. El sistema no "se cayó", **se colgó**.
- Con el sistema saturado la **CPU está baja**. La saturación se mide sobre el recurso que se agota: hilos ocupados contra hilos disponibles.

Nygard, *Release It!* (2018), cap. 4, *Blocked Threads*. Apunte: "Hilos bloqueados y pools agotados".

Reacciones en cadena y capacidades desbalanceadas



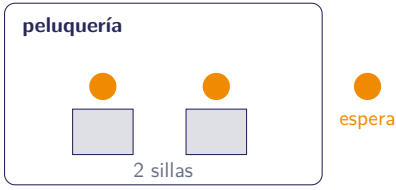
- **Reacción en cadena:** si la causa era una condición de carga, los sobrevivientes ahora son más propensos a fallar, y cada baja recarga a los demás. Como el defecto está en todos, solo se detiene corrigiéndolo.
- **Capacidades desbalanceadas:** un *front end* con 3000 hilos frente a un servicio con 450. Una promoción que lleve tráfico a ese camino lo desborda.

Nygard, *Release It!* (2018), cap. 4 (el ejemplo original usa ocho servidores). Apunte: "Reacciones en cadena".

Ley de Little: una peluquería

$$L = \lambda \cdot W$$

- λ: cuántos clientes **llegan** por hora.
- W: cuántas horas **se queda** cada uno.
- L: cuántos hay **adentro a la vez**.



Llegan 2 por hora y cada uno se queda 1,5 horas: $L = 2 \times 1{,}5 = \mathbf{3}$.

Clientes = pedidos · sillas = hilos del pool · tiempo adentro = tiempo que un pedido ocupa un hilo.

Con solo 2 sillas no da abasto, aunque nada esté "roto". Si cada corte se alarga, W sube y L sube **sin que llegue un solo cliente más**.

Little (1961). La analogía es de la cátedra. Apunte: "La ley de Little aplicada al pool de hilos".

La cuenta con un pool de hilos

Un servicio recibe $\lambda = 5$ pedidos por segundo y atiende con **8** hilos. Cada pedido hace 5 llamadas seguidas a una dependencia.

	W (por pedido)	$L = \lambda W$	¿Alcanzan 8 hilos?
Dependencia sana	$0{,}05$ s	$0{,}25$	sobran
Dependencia con 2 s por llamada	$5 \times 2 = 10$ s	$\mathbf{50}$	no: satura

Consecuencia

Saturado, el servicio sostiene $8/10 = 0{,}8$ pedidos por segundo, no 5. El resto espera en una cola. **Cambió W, no λ**: nadie se cayó.

El cliente ve *tiempo de residencia* = tiempo en cola + tiempo de procesamiento. Medir solo el segundo es la trampa.

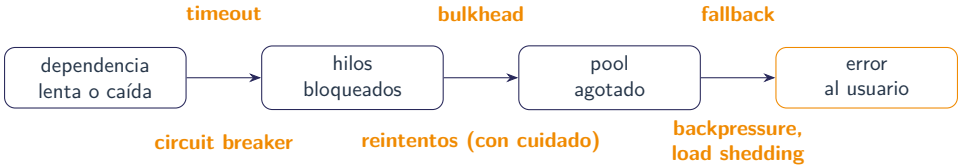
Little (1961); Nygard, *Release It!* (2018), cap. 9, *Demand Control*.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Resiliencia y sus herramientas

Dónde corta cada herramienta

Resiliencia: seguir funcionando ante fallas parciales, aunque sea degradado. No es que nada falle: es **contener** la propagación.

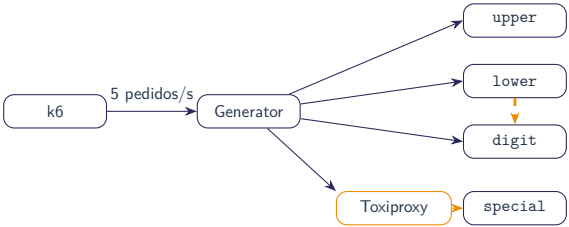


Timeout	corta la espera indefinida.
Reintento	recupera las fallas transitorias, a costa de más carga.
Circuit breaker	deja de insistir contra una dependencia que ya se sabe enferma.
Fallback	responde algo útil en lugar del error.
Bulkhead	impide que una parte agote los recursos que comparten las demás.
Backpressure, load shedding	frenan o rechazan la demanda mayor que la capacidad.

Apunte: "Resiliencia y sus herramientas".

El sistema de la demo

Un generador de contraseñas



- **k6** manda 5 pedidos por segundo a **ritmo fijo**: sigue mandando aunque el sistema se ponga lento. Corta cada pedido a los 10 s.
- El **Generator** arma cada contraseña llamando a cuatro servicios hijos, con un **pool de 8 hilos**.
- **Toxiproxy**, delante de `special`, inyecta las fallas sin tocar su código.
- **Dependencia oculta** (punteada): `lower` llama por dentro a `digit`.

Cómo se corrieron los experimentos: escenarios

Un **escenario** es la falla que Toxiproxy inyecta en `special`:

`baseline`: sin fallas. Es la línea de base.
`latency 2000`: cada llamada tarda 2 s más. La dependencia responde bien, pero lento.
`errors 0.1 / 0.5`: se corta la conexión en el 10 % o el 50 % de las llamadas.
`intermittent`: `special` se cae 20 s y vuelve 20 s, cinco veces. Con `fast` rechaza la conexión (falla rápida); con `hang` no responde (falla lenta).

Cada experimento duró de 3 a 4 minutos, con k6 enviando 5 pedidos por segundo. Los números de cada slide se midieron en Prometheus sobre la ventana del experimento. La traza y la tabla de la línea de base salen de una segunda corrida con la misma configuración.

Cómo leer las capturas

Cada **línea amarilla punteada** vertical marca el momento en que se aplicó una configuración: un escenario o un preset. Lo que está a su derecha corre con la configuración nueva.

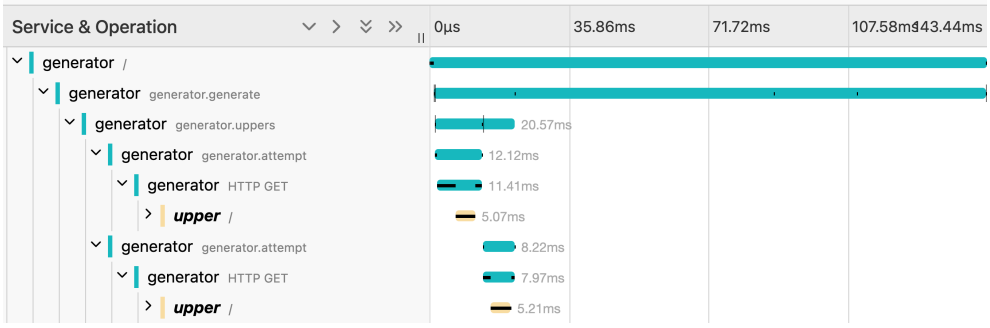
Cómo se corrieron los experimentos: presets

Un **preset** es la configuración de protecciones del Generator; se cambia en vivo.

Preset	Timeout	Reintentos	Circuit breaker	Fallback
`00-no-protection`	—	—	—	—
`01-timeout-only`	sí	—	—	—
`02-timeout-retry-naive`	sí	3, inmediatos	—	—
`03-timeout-retry-backoff-jitter`	sí	2, backoff + jitter	—	—
`04-cb-consecutive-fragile`	sí	—	3 fallas seguidas	sí
`04b-cb-consecutive-tolerant`	sí	—	10 fallas seguidas	sí
`05-cb-rate`	sí	—	tasa (50 / 20 / 50 %)	sí
`06-complete`	sí	2, backoff + jitter	tasa (20 / 10 / 50 %)	sí
`07-cb-no-fallback`	sí	—	tasa (50 / 20 / 50 %)	—

Tasa: ventana / mínimo de llamadas / umbral. Timeout hacia `special`: 100 ms. Breaker abierto: 10 s.
Fallback de la demo: si una llamada falla por *cualquier* motivo (timeout, error o rechazo del breaker), la contraseña se arma sin ese tipo de caracteres y se marca `degraded`. No es solo para el breaker abierto.

Fan-out: una traza de la línea de base



Recorte del comienzo de una traza. Para los caracteres en mayúscula, el Generator hace **dos llamadas** a `upper`: cada intento (`generator.attempt`) es un `HTTP GET` que llega al servicio `upper`. Después repite el patrón con `lower`, `digit` y `special`, uno tras otro. En la línea de base, cada pedido hizo en promedio **4,9 llamadas** a cada servicio hijo: toda política *por llamada* se multiplica por pedido.

Línea de base: latencia de cada dependencia

Latency per dependency over 5m (successful attempts, seen from the Generator)

Dependency ↑	p50	p95	p99	Calls per request (fan-out)
digit	8.5263 ms	23.8731 ms	42.5687 ms	4.87
lower	16.2424 ms	80.3883 ms	96.6140 ms	4.92
special	8.4499 ms	23.3115 ms	40.5134 ms	4.86
upper	8.8095 ms	23.6218 ms	38.7199 ms	4.81

- El p99 de `upper`, `digit` y `special` ronda los 40 ms. Es el dato para elegir los timeouts.
- `lower` tiene un p99 de 97 ms porque **incluye** la llamada interna a `digit`: medida desde el llamante, la cadena ya está adentro.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Experimento: una dependencia lenta

Sistemas Distribuidos

Sin protección: el pool se llena

Escenario: `latency 2000` · Preset: `00-no-protection` · línea amarilla: se aplica una configuración

Protecciones: ninguna

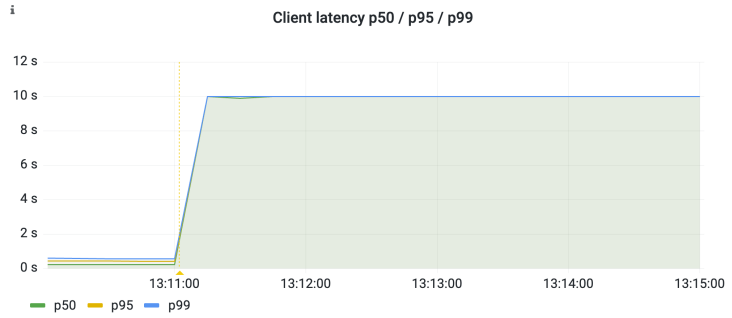


A los pocos segundos de la línea amarilla, los **8 hilos** quedan ocupados y no se liberan. Ley de Little con lo medido: cada pedido tardó $W \approx 10{,}4$ s, así que harían falta $L = 5 \times 10{,}4 \approx 52$ hilos. Se completaron 163 pedidos en 210 s: $0{,}78$ por segundo, que es $8/10{,}4$.

Sin protección: el usuario espera hasta rendirse

Escenario: `latency 2000` · Preset: `00-no-protection` · línea amarilla: se aplica una configuración

Protecciones: ninguna

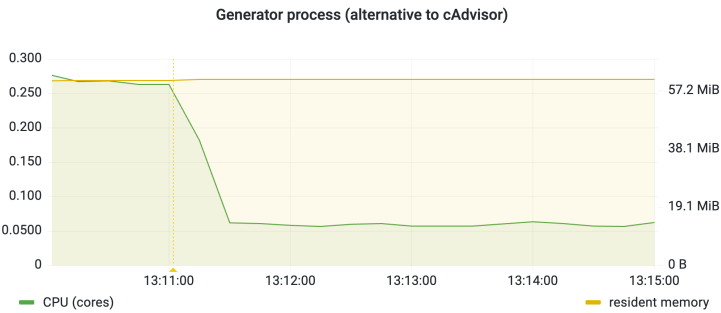


La latencia que ve k6 sube hasta **10 s**, que es su propio timeout: en la ventana medida, **todos** los pedidos del cliente terminaron por timeout. La lentitud de una dependencia se volvió la falla del servicio entero.

Sin protección: la CPU baja

Escenario: `latency 2000` · Preset: `00-no-protection` · línea amarilla: se aplica una configuración

Protecciones: ninguna

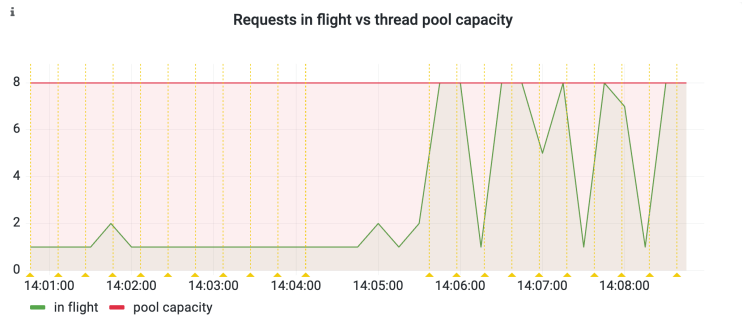


La CPU del Generator cae de 0,27 a 0,06 núcleos justo cuando el servicio deja de atender. Los hilos esperan, no calculan: un panel de CPU "en verde" es compatible con un servicio bloqueado.

Una respuesta lenta es peor que ninguna

Escenario: `intermittent fast → hang` · Preset: `00-no-protection` · línea amarilla: se aplica una configuración

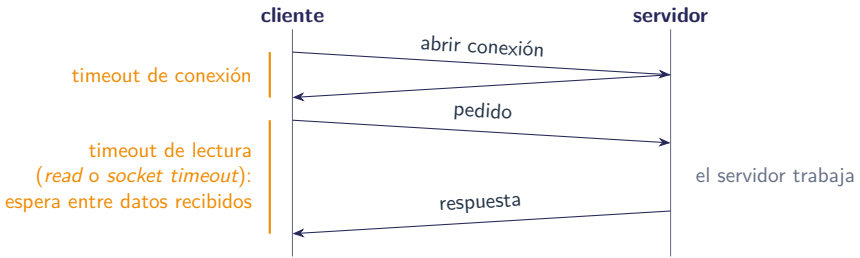
Protecciones: ninguna



`special` cae 20 s de cada 40. Con `fast` (hasta las 14:04) falla el 48 % de los pedidos, pero rápido: el pool casi no se usa. Con `hang` el pool **se llena en cada caída** y el 36 % termina por timeout del cliente.

Timeouts

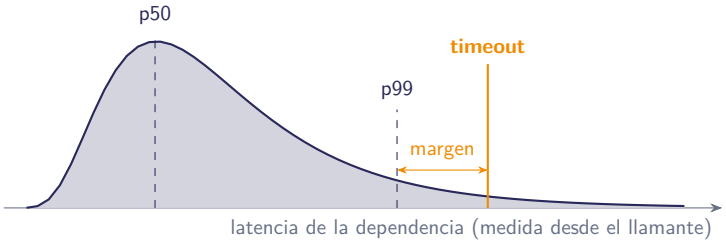
Timeout de conexión y timeout de lectura



- "Hope is not a design method": el timeout **no viene puesto**. `requests` (Python) no corta nunca sin el parámetro `timeout`; en Java, `URLConnection` con 0 espera para siempre.
- El de lectura mide el tiempo *entre* datos, no el total: para acotar el total hace falta un plazo global (*deadline*).

Nygard, *Release It!* (2018), cap. 5. Documentación de `requests` y de `java.net.URLConnection`. Apunte: "Timeouts".

Cómo elegir el valor: p99 + margen



- Con la **media**, el timeout cortaría muchas llamadas normales; con el **máximo**, casi nunca actuaría.
- En la demo: p99 de `special` ≈ 41 ms en la línea de base; los presets usan un timeout de lectura de **100 ms**.
- El percentil sale de buckets de histograma: es aproximado. Alcanza para calibrar, con margen.

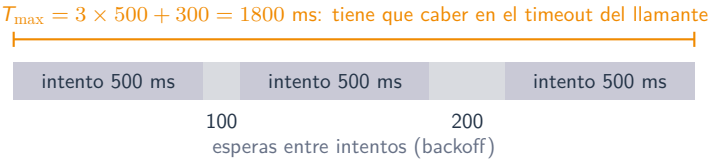
Apunte: "Cómo elegir el valor". Clase 10: percentiles.

Timeouts en una cadena de llamadas

Con reintentos, una llamada lógica puede tardar

$$T_{\max} = (1 + r) \cdot t + \sum_{i=1}^{r} e_i$$

t: timeout de cada intento; r: cantidad de **reintentos** ($1 + r$ intentos); e_i: espera antes del reintento i.



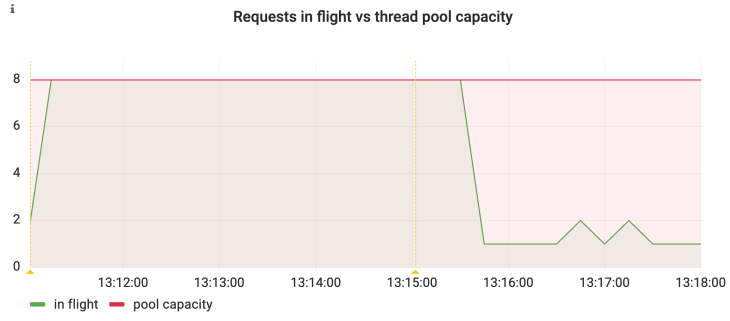
- Para el valor de partida no hace falta conocer la cadena: el p99 de B medido desde A ya incluye lo que B espera a C.
- Lo que la línea de base no muestra es el **peor caso interno**. Si el timeout de A es menor que el $T_{\max}$ de B, A abandona mientras B sigue trabajando.

Apunte: "Timeouts en una cadena de llamadas".

Experimento: solo timeouts

Escenario: `latency 2000` · Preset: `00-no-protection`, luego `01-timeout-only` · línea amarilla: se aplica una configuración

Protecciones: `00` ninguna; `01` solo timeout

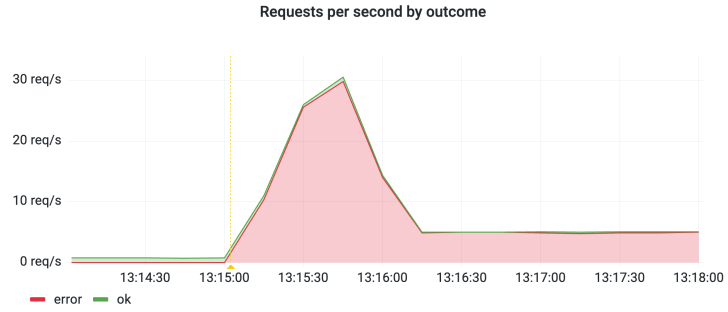


Al aplicar `01` (segunda línea amarilla), cada llamada a `special` se corta a los 100 ms y el hilo se libera: el pool baja de 8 a un promedio de 1,5 hilos ocupados. **Se terminó la saturación.**

Experimento: solo timeouts, pero todo falla

Escenario: `latency 2000` · Preset: `01-timeout-only` · línea amarilla: se aplica una configuración

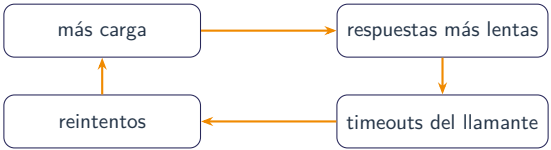
Protecciones: solo timeout



El **98 %** de los pedidos responde HTTP 500, ahora rápido (200 ms de media). Solo se salvan los pocos que no llegaron a pedir caracteres especiales. **Fallar rápido no es funcionar**: el timeout protege al llamante; al usuario lo protegen los mecanismos que siguen.

Reintentos

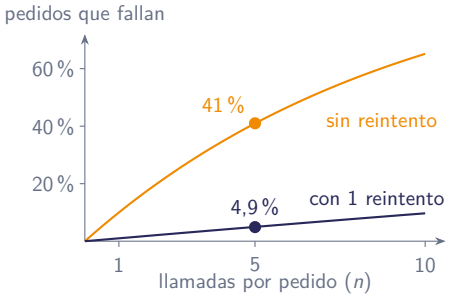
Cuándo destruyen: la retroalimentación



- El llamante abandona y reintenta "exactly when it hurts the worst".
- Un *backend* atiende 10 000 consultas por segundo y recibe 10 100: rechaza 100, que vuelven como reintentos (10 200), luego 300 rechazos… Bajar la carga al nivel previo puede no alcanzar para salir.
- Ayudan frente a fallas **transitorias**. Solo operaciones **idempotentes**, y solo errores reintentables (timeouts, conexión, 5xx; no 4xx).

Nygard, *Release It!* (2018), caps. 5 y 9; Beyer et al., *Site Reliability Engineering* (2016), cap. 22.

La cuenta de la amplificación



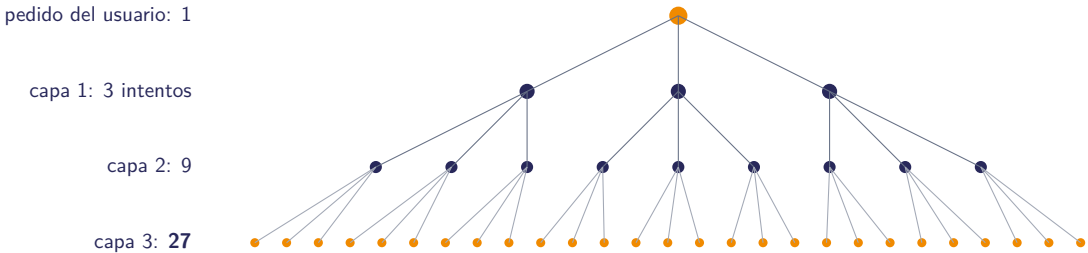
Un pedido hace n llamadas y cada una falla con $p = 10\,\%$:

$$P(\text{falla del pedido}) = 1 - (1-p)^n$$

- $n = 5$: $1 - 0{,}9^5 \approx \mathbf{41}\,\%$.
- Con un reintento la llamada falla si fallan **los dos** intentos: $p^2 = 1\,\%$, y el pedido, $1 - 0{,}99^5 \approx \mathbf{4{,}9}\,\%$.
- Costo: $1 + p = 1{,}1$ intentos por llamada. Pero ante una caída, $p \to 1$: si se permiten r reintentos por llamada (acá $r = 1$), cada llamada pasa a ser $1 + r$ intentos.

Apunte: "La cuenta de la amplificación".

Reintentos en capas

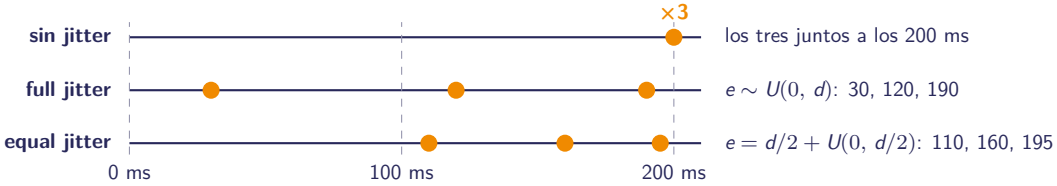


- Cada capa reintenta y los factores **se multiplican**: $3^3 = 27$ llamadas por pedido. Con 3 reintentos (4 intentos) en tres capas, $4^3 = 64$ intentos sobre la base de datos.
- Decidir **en qué capa** se reintenta. Y un rechazo del breaker **nunca** se reintenta.

Beyer et al., *Site Reliability Engineering* (2016), cap. 22.

Backoff exponencial y jitter

$d_n = \min(\text{tope},\ \text{base} \cdot m^n)$. Base 100 ms, $m = 2$: el primer reintento espera 100 ms, el segundo 200 ms. Pero si todos esperan lo mismo, **reintentan juntos**.



Tres clientes fallan a la vez y esperan su segundo reintento ($d = 200$ ms). *Full jitter* dispersa más; *equal jitter* garantiza una espera mínima. El efecto solo se nota con **muchos** clientes.

Brooker, *Exponential Backoff and Jitter*, AWS Architecture Blog (2015). Nygard (2018), cap. 4, *Dogpile*.

Presupuesto de reintentos

Limitar los intentos *por pedido* no alcanza: si la dependencia entera está sobrecargada, todos los pedidos agotan sus intentos y la carga casi se triplica.

Dos límites (Google)

- **Por pedido:** como máximo 3 intentos.
- **Por cliente:** reintentar solo mientras los reintentos sean menos del 10 % de los pedidos que envía.

- Solo con el primero, los pedidos crecen hasta algo menos del triple; con los dos, en el caso general, 1,1 veces.
- El segundo se ajusta solo: si fallan muchos pedidos, el problema no es transitorio y se deja de reintentar.
- Variante simple: un tope por proceso, por ejemplo 60 reintentos por minuto.

Beyer et al., *Site Reliability Engineering* (2016), caps. 21 y 22. Apunte: "Presupuesto de reintentos".

Caso Pokémon GO

- Se probó la carga hasta 5 veces la estimación más optimista. El lanzamiento real tuvo **casi 50 veces** esa estimación.
- La app reintentaba con "a single immediate retry, followed by constant backoff".
- Los *backends* devolvían errores rápidos: eso **sincronizó** los reintentos. Picos de hasta **20 veces** el máximo global anterior (*thundering herd*).
- Mitigación de fondo: **jitter** y *backoff* exponencial truncado en los clientes.

Lección

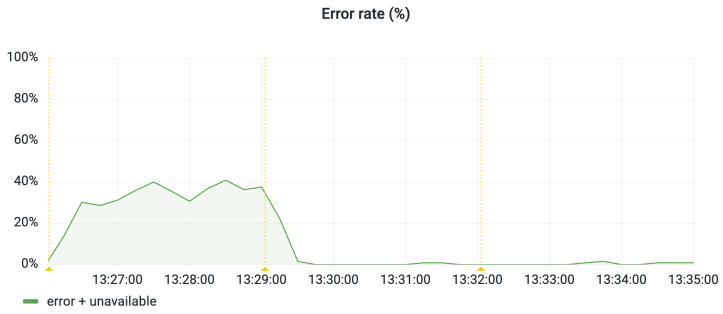
Muchos clientes independientes, con reintentos sin dispersión, se vuelven **una sola ola**.

Bethea et al., en Beyer et al., *The Site Reliability Workbook* (2018), cap. 11.

Experimento: reintentos contra 10 % de errores

Escenario: `errors 0.1` · Preset: 00 → 02 → 03 · línea amarilla: se aplica una configuración

Protecciones: 00 ninguna · 02 timeout + 3 reintentos inmediatos · 03 timeout + 2 reintentos con backoff y jitter

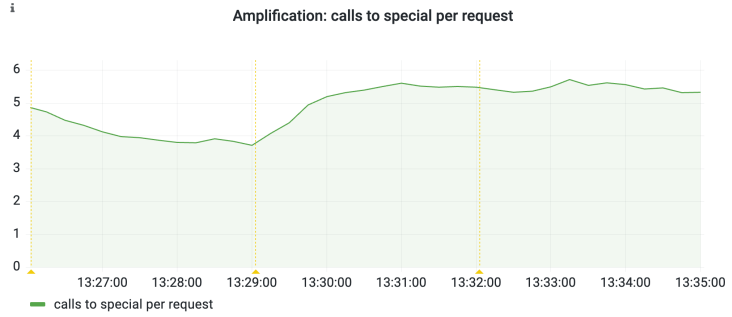


Sin protección falla el **36,8 %** de los pedidos (la cuenta con $n = 4{,}9$ da 40 %; como n varía de pedido en pedido, se mide algo menos). Con 3 reintentos inmediatos (02) falla el **0,14 %**; con 2 reintentos con backoff y jitter (03), el **0,55 %**.

Experimento: el costo de los reintentos

Escenario: `errors 0.1` · Preset: 00 → 02 → 03 · línea amarilla: se aplica una configuración

Protecciones: 00 ninguna · 02 timeout + 3 reintentos inmediatos · 03 timeout + 2 reintentos con backoff y jitter



Con reintentos, cada pedido hace **5,4 llamadas** a `special`, contra 4,9 en la línea de base: un 10 % más, el $1 + p$ de la cuenta (sin protección son 3,9: el pedido se corta en la primera falla). Entre 02 y 03 **no hay diferencia visible**: con un solo cliente no hay sincronización que romper.

UNIVERSIDAD AUSTRAL

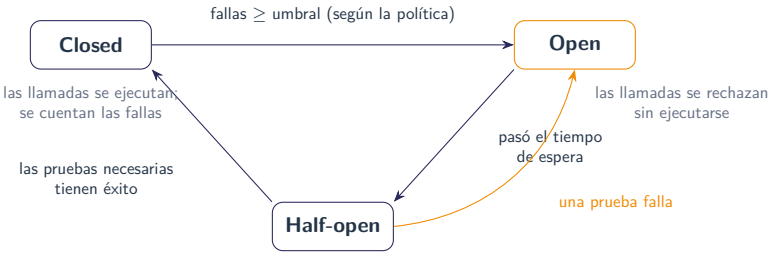
INGENIERÍA

Departamento de Computación

Circuit breaker

Sistemas Distribuidos

La máquina de estados



Qué es

Un componente que "falla primero" para que el sistema no falle: existe para **prevenir** operaciones, no para reejecutarlas.

Nygard, *Release It!* (2018), cap. 5, *Circuit Breaker*.

Las decisiones de configuración

Qué cuenta como falla: timeouts, errores de conexión y 5xx. **No** los 4xx ni los errores de validación.

Con timeouts: sin timeout, la llamada lenta no termina y el breaker **no ve** la falla más peligrosa.

Cuándo abrir: fallas consecutivas o tasa de fallas en ventana (siguientes slides).

Cuánto tiempo abierto: muy corto, muchas pruebas fallidas; muy largo, degradación de más.

Half-open: cuántas pruebas a la vez y cuántos éxitos para cerrar.

Alcance: un breaker **por dependencia** y por proceso (en la demo, `generator-special` y los otros tres).

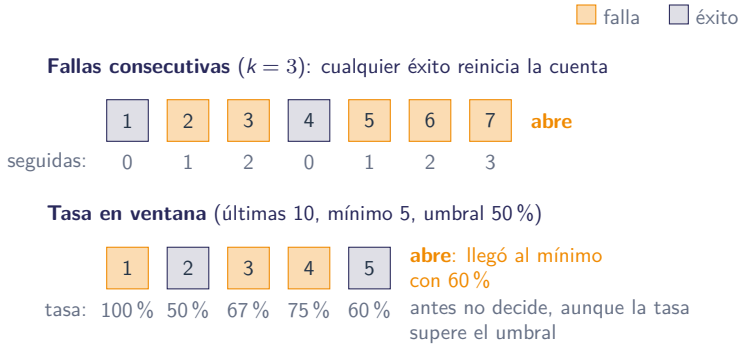
Con reintentos: cada intento cuenta; **un rechazo no se reintenta**.

Implementación: *thread-safe*, con el *lock* solo sobre el estado, **nunca** sobre la llamada de red: si no, serializa todas las llamadas.

Observabilidad: los cambios de estado se registran y se grafican.

Nygard, *Release It!* (2018), cap. 5. Apunte: "Circuit breaker".

Cuándo abrir: dos políticas, paso a paso



En los dos casos, abierto se rechaza todo durante el tiempo de espera; después, en *half-open*, una llamada de prueba decide si cierra o vuelve a abrir.

Apunte: "Cuándo abrir: fallas consecutivas contra tasa de fallas" (tablas paso a paso).

El dilema de las fallas consecutivas

Con probabilidad de falla p, llamadas esperadas hasta ver k fallas seguidas:

$$E(k, p) = \frac{1 - p^k}{(1 - p)\, p^k}$$

	$p = 10\,\%$	$p = 50\,\%$
$k = 3$	≈ 1110 llamadas	14 llamadas
$k = 10$	$\approx 1{,}1 \times 10^{10}$ (nunca)	≈ 2046 llamadas

Ningún k sirve para las dos cosas

Tolerar el 10 % y detectar rápido el 50 %. Las fallas consecutivas **mezclan tasa de error con volumen de tráfico**: a 100 llamadas por segundo, con $k = 3$ y 10 s abierto, el breaker abre cada ≈ 11 s y está abierto cerca del 47 % del tiempo con un 10 % de errores.

Cuenta propia (rachas en ensayos independientes de Bernoulli). Apunte: “El dilema”.

La solución: tasa de fallas en una ventana

Ventana deslizante con las últimas N llamadas: abre cuando la **proporción** de fallas llega al umbral, pero **solo tras un mínimo** de llamadas.

- Preset `05-cb-rate`: ventana 50, mínimo 20, umbral 50 %.
- Con $p = 10\,\%$: abrir exige 25 o más fallas en 50. Probabilidad del orden de 10^{-12}: **no abre**.
- Con $p = 50\,\%$: la primera evaluación, con 20 llamadas, ya tiene cerca de un 59 % de probabilidad de ver 10 o más fallas. Abre en decenas de llamadas, **sin importar el volumen**.

Compromiso

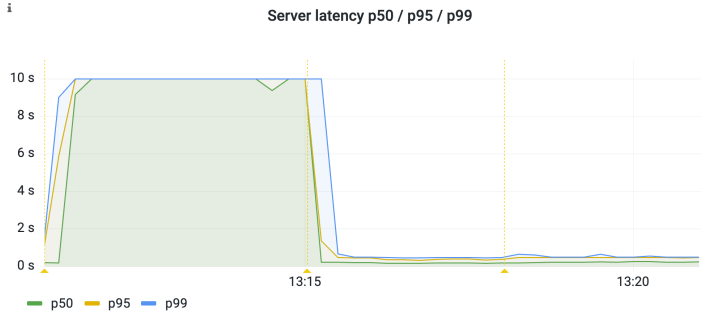
Ventana chica: rápida pero ruidosa. Ventana grande: estable pero lenta. Variante: abrir también por **llamadas lentas**.

Cuenta propia (distribución binomial). Apunte: "Política de tasa de fallas en ventana".

Experimento: breaker y fallback contra la dependencia lenta

Escenario: `latency 2000` · Preset: 00 → 01 → 06 · línea amarilla: se aplica una configuración

Protecciones: 00 ninguna · 01 timeout · 06 timeout + reintentos + breaker por tasa + fallback

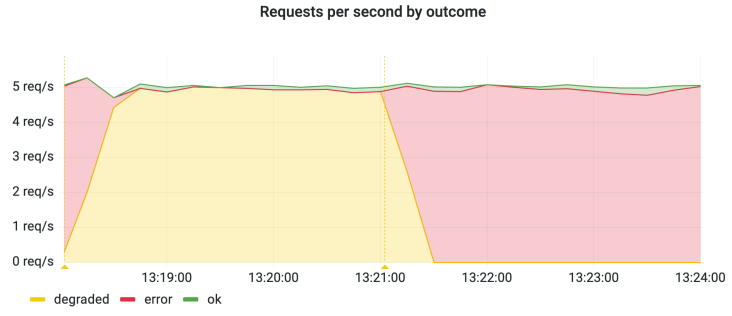


La latencia cae con 01 y sigue baja con 06 (p99 del cliente: **0,67 s**). La diferencia es el resultado: con 06 el breaker se abre, las llamadas se rechazan sin esperar y hay **cero errores**; el 98 % de las contraseñas sale sin caracteres especiales (`degraded`).

Experimento: el mismo breaker, sin fallback

Escenario: `latency 2000` · Preset: 06 → 07 · línea amarilla: se aplica una configuración

Protecciones: 06 timeout + reintentos + breaker + fallback · 07 timeout + breaker, sin fallback

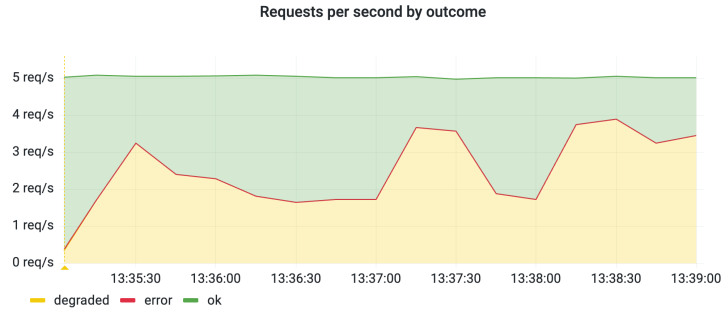


Con 06 los pedidos salen degradados (amarillo); con 07 el breaker abre igual, pero sin fallback salen como **error** (rojo): el 98 %. El breaker protege al *sistema*; que el usuario reciba algo útil es una decisión de producto, el *fallback*.

Experimento: 10 % de errores con fallas consecutivas

Escenario: `errors 0.1` · Preset: `04-cb-consecutive-fragile` · línea amarilla: se aplica una configuración

Protecciones: timeout + breaker por 3 fallas seguidas + fallback; sin reintentos

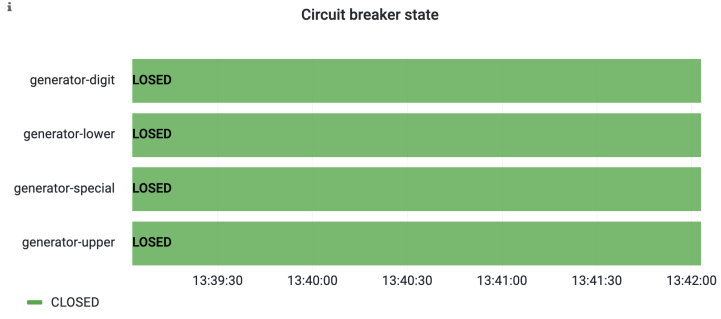


Un 10 % de errores que el sistema toleraba abrió el breaker **5 veces en 210 s** (la cuenta da una racha de 3 fallas cada ≈ 1110 llamadas, cerca de un minuto a este ritmo). Mientras está abierto, **todas** las llamadas a `special` se rechazan y van al fallback: salió degradado el **53 %** de los pedidos.

Experimento: 10 % de errores con tasa en ventana

Escenario: `errors 0.1` · Preset: `05-cb-rate` · línea amarilla: se aplica una configuración

Protecciones: timeout + breaker por tasa (ventana 50, mínimo 20, 50 %) + fallback; sin reintentos

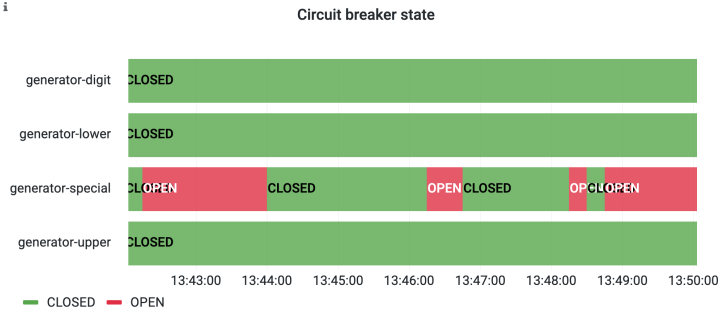


El breaker de `special` **no abre nunca**: la tasa (10 %) está lejos del umbral (50 %). Aun así sale degradado el 38 % de los pedidos: en la demo el fallback actúa ante **cualquier** llamada que falla, no solo con el breaker abierto. Se degradan solo los pedidos con alguna conexión cortada.

Experimento: 50 % de errores, las tres configuraciones

Escenario: `errors 0.5` · Preset: 04 → 04b → 05 · línea amarilla: se aplica una configuración

Protecciones: timeout + breaker + fallback; el breaker abre con 3 fallas seguidas (04), con 10 (04b) o por tasa (05)

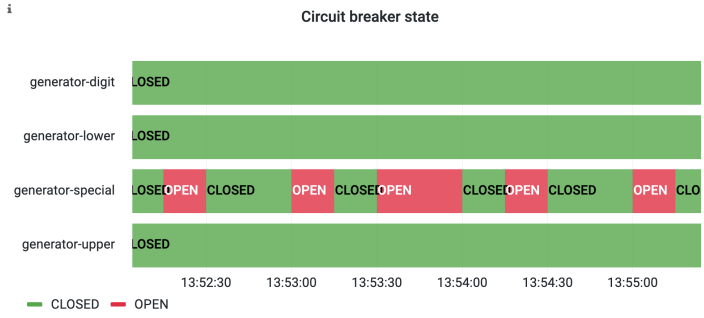


04 ($k = 3$, desde 13:42) abre enseguida. 04b ($k = 10$, desde 13:44) tarda **más de 2 minutos** en abrir por primera vez y deja pasar **1867** llamadas fallidas en 210 s. 05 (tasa, desde 13:48) abre enseguida: 31 llamadas fallidas en 90 s, igual que 04. **El dilema, medido**.

Experimento: caída intermitente

Escenario: `intermittent fast` · Preset: `06-complete` · línea amarilla: se aplica una configuración

Protecciones: timeout + reintentos + breaker por tasa + fallback



El breaker de `special` se abre en cada caída y se cierra cuando `special` vuelve. Hubo **0 errores**: durante las caídas los pedidos salieron degradados (49 %). El estado *half-open* dura lo que tarda una prueba (milisegundos) y no llega a verse con el muestreo del panel, cada 5 s.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Degradación, bulkheads y control de la demanda

Sistemas Distribuidos

Degradación elegante (graceful degradation)

normal

```
{"password": "k7#Qa2!mXp",
 "degraded": []}
```

sin `special`: HTTP 200, degradada

```
{"password": "k7aQa2mXpR",
 "degraded": ["special"]}
```

- El *fallback* es una **decisión de producto**: caché, respuesta genérica, servicio secundario. Se decide con los interesados, de antemano.
- Tiene que ser **explícito** (el campo `degraded`) y **más barato** que el camino normal.
- A veces no es aceptable: una contraseña sin caracteres especiales puede violar la política de contraseñas. Degradar pasa a ser responder algo incorrecto; corresponde fallar explícito.

Nygard, *Release It!* (2018), cap. 5. Apunte: "Degradación elegante".

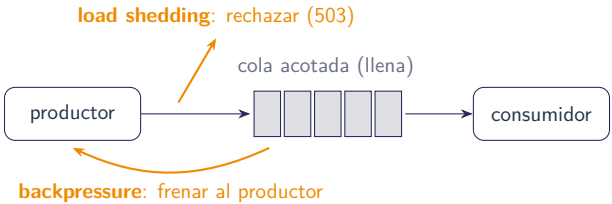
Bulkheads: compartimentos estancos



- Como los mamparos de un barco: el agua inunda un compartimento y el barco no se hunde. "Salvar parte del barco".
- Formas: hilos por dependencia (y un pool reservado para administración), procesadores, instancias dedicadas a clientes críticos, zonas y regiones.
- **Costo**: capacidad ociosa. Hay que buscar los límites naturales donde la partición conviene.

Nygard, *Release It!* (2018), cap. 5, *Bulkheads*. Apunte: "Bulkheads".

Backpressure y load shedding



- "Every performance problem starts with a queue backing up somewhere." Sin límite, la cola crece y, por Little, también el tiempo de respuesta.
- **Backpressure** desacelera al productor (TCP lo hace con su ventana). Sirve con productores conocidos.
- **Load shedding** rechaza lo que no se va a completar a tiempo, lo antes posible: un rechazo rápido es mejor que un timeout lento. Ejemplo: un servicio de video en vivo descarta los cuadros que ya llegarían tarde (ejemplo de la cátedra).

Nygard, *Release It!* (2018), caps. 5 y 9. Apunte: "Backpressure y load shedding".

Dos mecanismos sanos que interactúan mal: Dressy

- Caso ilustrativo (ficticio) del Workbook: tres regiones detrás de un balanceador.
- Se activó *load shedding* por CPU. La región A empieza a rechazar 10 %, 20 %, 50 % de los pedidos; su CPU queda constante.
- El balanceador estima el costo por pedido con la CPU: ve la región A como diez veces más eficiente y le manda más tráfico (400 pedidos por segundo contra 40 en B y C).

Lección

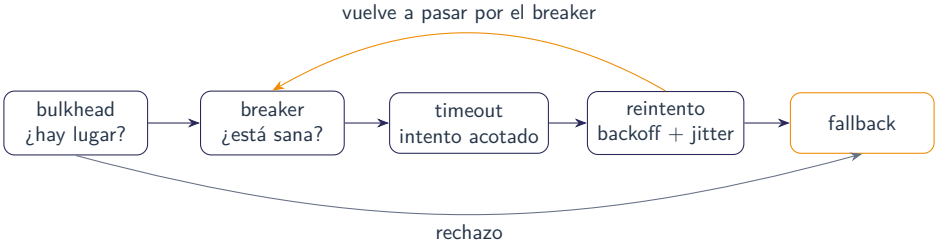
"The load balancer didn't know that the 'efficient' requests were errors." Cada mecanismo debe examinarse junto con los demás, no por separado.

Bethea et al., *The Site Reliability Workbook* (2018), cap. 11.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Combinar mecanismos

El orden dentro de una llamada



- Cada mecanismo ahorra recursos al siguiente: bulkhead y breaker evitan intentos inútiles, el timeout acota los que se hacen, los reintentos recuperan lo transitorio sin saltarse las protecciones, el fallback decide qué hacer con el resto.
- Más patrones no es mejor: "Count of patterns applied" nunca es una buena métrica. Cada uno agrega configuración y modos de falla propios.

Nygard, *Release It!* (2018), cap. 5. Apunte: "Combinar mecanismos".

Caso: la excepción que dejó en tierra a una aerolínea



- Una excepción al cerrar la consulta impedía devolver la conexión al pool. Tras unas cuarenta llamadas, el pool se agotó.
- Todos los hilos de CF quedaron esperando una conexión, y los 40 hilos de cada servidor de los quioscos de *check-in* (las terminales de autoservicio de los aeropuertos), esperando a CF por RMI, sin timeout.
- El monitoreo solo miraba una página de estado: CF parecía sano.

Dónde se cortaba

Timeout en el pool y en el cliente · breaker en el cliente · bulkheads en CF · fallback decidido con el negocio · medir desde el cliente.

Nygard, *Release It!* (2018), caps. 2 y 3. Apunte: "Caso de estudio" (relato completo).

Cierre

Modelo del Curso: Resiliencia en $D = (P, F, G, S, T)$

P (Problema): Impedir que la falla o la lentitud de una dependencia se propague al resto del sistema.

F (Factores): Fallas parciales, dependencias lentas, pools finitos de hilos, fan-out, reintentos que amplifican la carga.

G (Garantías): Contención del daño, latencia acotada, recuperación automática, degradación explícita.

S (Solución): Timeouts calibrados con percentiles + reintentos con backoff y jitter + breaker por dependencia (tasa de fallas) + fallback decidido con el negocio.

T (Trade-offs): Degradación de la respuesta, capacidad ociosa (bulkheads), complejidad de configuración, falsos positivos del breaker, interacción entre mecanismos.

Cierre

Idea central de cátedra

La resiliencia distribuida es control de propagación de falla, no negación de la falla.

- Sin protección, una dependencia lenta saturó el pool y tumbó el servicio entero.
- Los timeouts cortaron la saturación; los reintentos recuperaron lo transitorio; el breaker con fallback convirtió errores en respuestas degradadas.
- La política del breaker importa: con fallas consecutivas, ningún umbral sirve para el 10 % y el 50 % a la vez.